FASE 7 · BACKEND Y ORQUESTACIÓN

Conexión real con OpenAI Agents SDK

Arquitectura multiagente, generación de 13 apartados, persistencia en MongoDB y validación integral frontend–backend.

13 / 13	5 + 1	65 / 65	100%
apartados generados	agentes + subagente	pruebas aprobadas	flujo completado

Resultado principal

Ágora dejó de simular los componentes. El backend ejecutó una generación real con el SDK oficial, guardó el progreso en MongoDB y presentó los borradores en el workspace de revisión humana.

Proyecto de validación: AgroTech · piloto hidropónico inteligente
Fecha de cierre técnico: 18 de agosto de 2026
Rama: feat/Fase7_Backend_Primera_conexion_frontend_backend
Commit solicitado: feat/ConexiónAgentesSDK

1. Alcance y resultado alcanzado

La Fase 7 conecta el frontend React con una API NestJS persistente y añade la generación multiagente real. El flujo autenticado recibe el contexto confirmado del proyecto, coordina especialistas, persiste avances parciales y expone trece componentes editables para revisión humana.

> **Criterio de éxito** Un proyecto confirmado debe producir los 13 apartados con contenido utilizable —no instrucciones sobre qué escribir—, conservarlos en MongoDB y permitir su revisión desde la interfaz.

Arquitectura implementada

Frontend React	→	**API NestJS**	→	**ProjectOrchestrator**
JWT + ownership		MongoDB Atlas		5 especialistas

El adaptador híbrido del frontend consume los endpoints reales de componentes. El backend valida propietario y estado del proyecto, inicia el orquestador y actualiza componentes, progreso y salidas de agentes después de cada especialidad.

Decisiones de seguridad

Control	Aplicación
Clave de OpenAI	Solo en el backend mediante variable de entorno; nunca se expone al navegador.
Autorización	JWT y consulta por ownerId para impedir acceso cruzado entre proyectos.
Trazas	Agents SDK con traceIncludeSensitiveData=false y metadatos mínimos.
Persistencia	MongoDB guarda estado, componentes, salidas por agente y versión de generación.

2. Composición de agentes y apartados

Especialista	Apartados generados	Comportamiento
Investigación	Marco teórico; estado del arte; planteamiento; antecedentes; bibliografía	Recibe un dossier del subagente documental y redacta salida estructurada con fuentes.
Ideación	Actividades; objetivos SMART; cronograma	Conecta objetivos, actividades, productos, indicadores y secuencia temporal.
Impacto	Beneficiarios; riesgos; impactos esperados	Caracteriza grupos, riesgos, mitigaciones, evidencia e impactos verificables.
Presupuesto	Presupuesto	Vincula rubros con actividades y deja valores pendientes cuando faltan datos.
Metodología	Metodología	Integra las salidas previas en fases, procedimientos, roles, evidencias y controles.

Patrón interno del agente de investigación

El uso simultáneo de búsqueda web y JSON estricto produjo texto no serializado. Se corrigió con dos pasos: (1) subagente documental con búsqueda web; (2) especialista redactor sin herramientas, responsable del contrato estructurado.

Controles de calidad de salida

Los contratos Zod obligan a devolver todos los campos del dominio, enums controlados, identificadores y estructuras trazables. Las instrucciones del sistema exigen párrafos finales, argumentados y conectados con el contexto, y prohíben respuestas evasivas como “deberá desarrollarse”.

La metodología dispone de un presupuesto de salida mayor porque integra todas las dependencias. Cuando una ejecución falla después de agentes exitosos, el proyecto conserva esas salidas y reanuda desde el primer especialista pendiente.

3. Resultados de la generación real

#	Apartado	Agente	Caracteres	Estado
1	**Marco teórico**	Investigación	7.214	Generado
2	**Estado del arte**	Investigación	749	Generado
3	**Planteamiento del problema**	Investigación	894	Generado
4	**Antecedentes**	Investigación	726	Generado
5	**Bibliografía**	Investigación	504	Generado
6	**Actividades**	Ideación	6.840	Generado
7	**Objetivos SMART**	Ideación	1.486	Generado
8	**Cronograma**	Ideación	533	Generado
9	**Beneficiarios**	Impacto	2.425	Generado
10	**Riesgos**	Impacto	4.664	Generado
11	**Impactos esperados**	Impacto	5.421	Generado
12	**Presupuesto**	Presupuesto	1.031	Generado
13	**Metodología**	Metodología	9.939	Generado

Volumen validado

Los 13 componentes suman 42.426 caracteres. El marco teórico contiene 992 palabras y la metodología 1.340 palabras según los contadores visibles del frontend.

La respuesta obtenida ya no es un listado de recomendaciones. El marco teórico desarrolla ejes conceptuales, relación con el proyecto y aplicación; la metodología describe fases, procedimientos, trazabilidad, riesgos, control de calidad y transferencia.

4. Evidencia visual · generación completada



Figura 1. Pantalla de generación: progreso al 100% y 13 de 13 apartados finalizados.

5. Evidencia visual · workspace de revisión

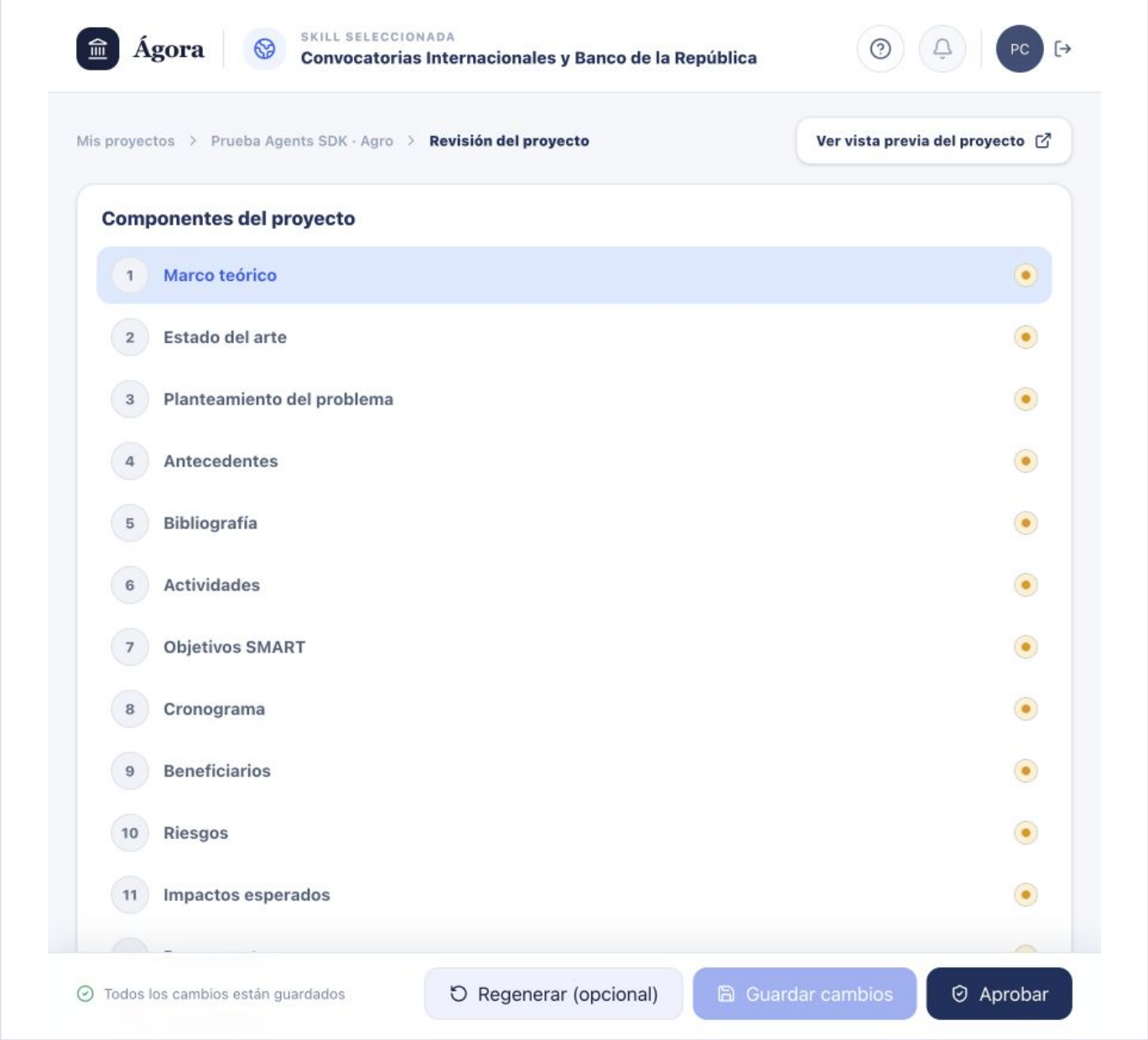


Figura 2. Los trece componentes aparecen en el workspace y quedan pendientes de revisión humana.

El estado “pendiente” es intencional: la IA genera borradores, pero no aprueba contenido en nombre del usuario. La aprobación y el ensamblaje permanecen bajo control humano.

6. Evidencia visual · calidad del marco teórico

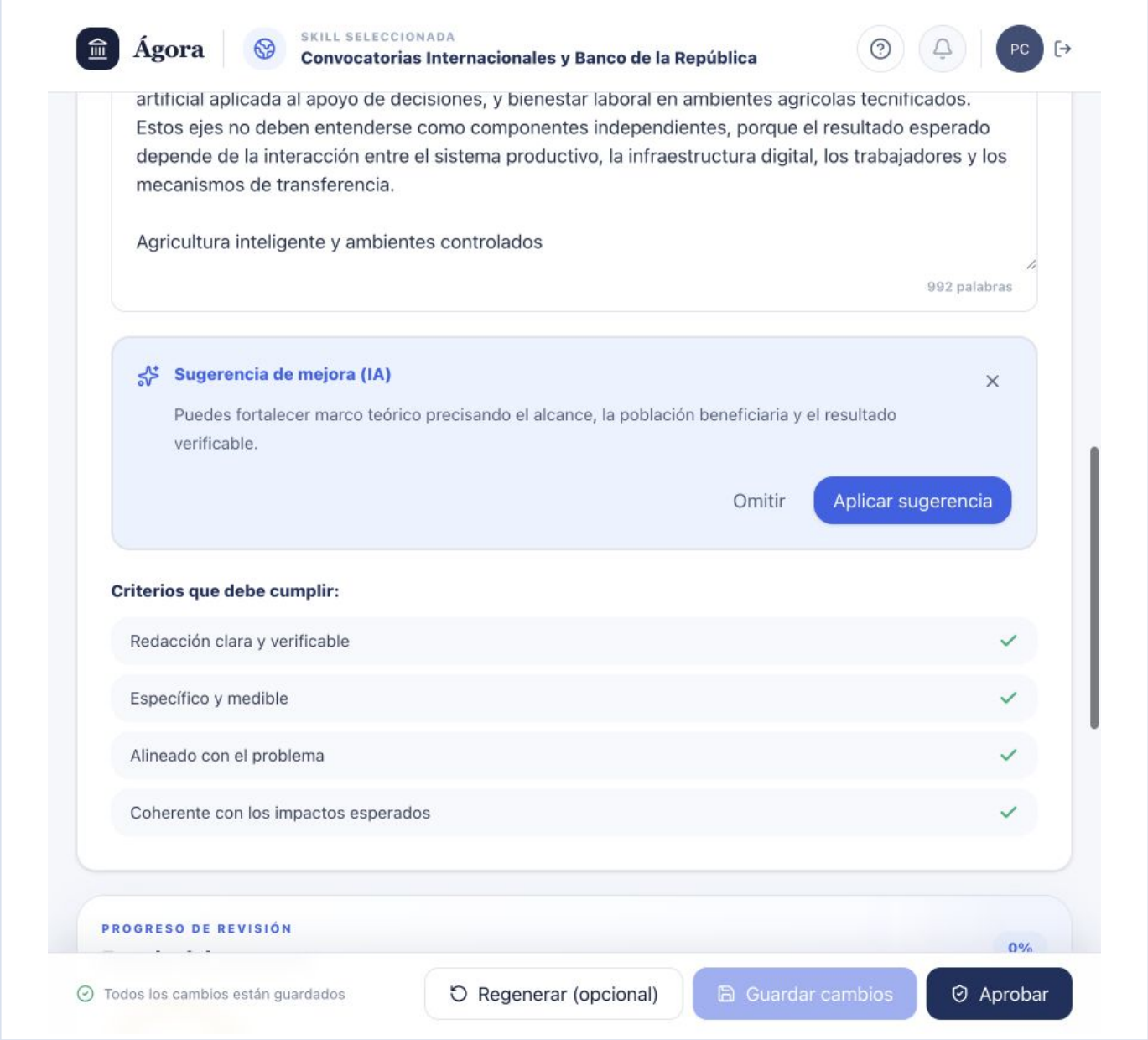


Figura 3. Marco teórico con 992 palabras, sugerencia asistida y criterios automáticos satisfechos.

El texto visible conecta agricultura inteligente, hidroponía, IoT, inteligencia artificial, economía circular, ergonomía y transferencia tecnológica con el piloto AgroTech.

7. Evidencia visual · metodología generada



Figura 4. Metodología generada con 1.340 palabras y contenido editable.

La reanudación inteligente permitió generar únicamente este último apartado después de una salida truncada, conservando los doce componentes ya guardados.

8. Pruebas ejecutadas y resultados

Capa	Validación	Resultado
Backend	Jest: 11 suites / 38 pruebas	Aprobado
Backend	TypeScript typecheck	Aprobado
Backend	ESLint sin warnings	Aprobado
Backend	Compilación NestJS	Aprobado
Frontend	Vitest: 8 archivos / 27 pruebas	Aprobado
Frontend	TypeScript build	Aprobado
Frontend	ESLint sin warnings	Aprobado
Frontend	Vite build de producción	Aprobado
Integración	Frontend 5176 → API 3000 → MongoDB Atlas	Aprobado
Integración	OpenAI Agents SDK → 13 componentes	Aprobado
Interfaz	Generación 100% y workspace de revisión	Aprobado

Total automatizado 65 pruebas aprobadas: 38 en backend y 27 en frontend. Los builds de producción también finalizaron correctamente.

Incidencias encontradas durante la prueba real

Incidencia	Causa	Corrección
429 de OpenAI	Razonamiento alto y búsqueda extensa superaban el TPM.	Presupuesto por agente, razonamiento medio y reintentos con espera indicada por OpenAI.
Salida no JSON en investigación	Búsqueda web y salida estructurada competían en el mismo turno.	Subagente documental separado del redactor estructurado.
Metodología truncada	El último agente integra cuatro salidas y requiere mayor respuesta.	10.000 tokens de salida y reanudación desde el agente pendiente.
Espera HTTP prolongada	La generación completa puede superar el timeout de un cliente síncrono.	Persistencia progresiva; queda recomendado convertir el disparo en job asíncrono.

9. Conclusiones y próximos pasos

Conclusión

La conexión del Agents SDK quedó funcional y demostrada con datos reales. Ágora coordina especialistas, usa evidencia externa de forma controlada, persiste resultados parciales, se recupera de fallos y entrega 13 borradores editables para revisión humana.

Logros cerrados

- Integración oficial de @openai/agents en el backend, con trazas y seguridad configuradas.
- Cinco especialistas alineados con la composición definitiva de 13 apartados.
- Subagente documental para separar investigación web de redacción estructurada.
- Persistencia de componentes, salidas por agente y versión de generación en MongoDB.
- Reanudación desde el agente fallido sin perder ni regenerar resultados correctos.
- Frontend conectado al endpoint real y evidencia visual del 100% de generación.
- Batería de 65 pruebas aprobadas, builds, lint y typecheck satisfactorios.

Recomendaciones para la siguiente iteración

- Convertir la generación en un job asíncrono con identificador, polling o eventos para evitar timeouts de clientes HTTP.
- Añadir una rúbrica automática de calidad por apartado y una acción de regeneración focalizada.
- Sustituir el prompt provisional de metodología cuando se entregue la versión maestra definitiva.
- Incorporar pruebas end-to-end de autenticación, creación, generación, revisión y ensamblaje en CI.
- Mantener revisión humana obligatoria antes de aprobar, ensamblar o exportar el documento final.

Estado de entrega

Código listo para commit en la rama existente de Fase 7. No se incluyen archivos .env, credenciales ni copias duplicadas ajenas al alcance.